

- System load
- Network configuration
- Ping checks
- Customised TCP port checks
- Watching files for changes
- Scanning of log files

Pycap (Packet Capture Library)

Pycap is a Python wrapper with object-oriented integration for libcap. It can be installed easily:

```
$ pip install pycap
```

Using Python pcap, the packets can be captured with the following few lines of code:

```
>>> import pcap
>>> for ts, pkt in pcap.pcap():
print ts, 'pkt'
```

LinkChecker

Recursive and deep checking of server pages can be done using the LinkChecker library in Python. Site crawling is made easy with its features for integrating regular expressions and filtering using LinkChecker. The output can be generated in multiple formats including HTML, XML, CSV, SQL or simply the sitemap graph.

WebScraping

Python is used by researchers and practitioners for collecting live data for research and development. For example, we can fetch live records of the stock market, the price of any product from e-commerce websites, etc. Data collected in this way forms the foundation of Big Data analytics. If a researcher is working on Big Data analysis, the live data can be fetched using a Python script and can be processed based on the research objectives.

Here is a code snippet written in Python to fetch live stock exchange data from the website *timesofindia.com*:

```
from bs4 import BeautifulSoup
import urllib.request
from time import sleep
from datetime import datetime
def getnews():
    url = "http://timesofindia.indiatimes.com/business"
    req = urllib.request.urlopen(url)
    page = req.read()
    scraping = BeautifulSoup(page)
    price = scraping.findAll("span", attrs={"class": "red14px"})
[0].text
    return price
with open("bseindex.out", "w") as f:
    for x in range(2,100):
```

```
sNow = datetime.now().strftime("%I:%M:%S%p")
f.write("{0}, {1} \n ".format(sNow, getnews()))
sleep(1)
```

Fetching live data from social media

In the same way, Twitter live feeds can be fetched using Python APIs. Using a Twitter developer account, a new app can be created and then a Python script can be mapped with the Twitter app, as follows:

```
from tweepy import Stream
from tweepy import OAuthHandler
from tweepy.streaming import StreamListener
```

#setting up the keys

```
consumer_key = 'XXXXXXXXXXXXXXXXXXXXX'
consumer_secret = 'XXXXXXXXXXXXXXXXXXXXX'
access_token = 'XXXXXXXXXXXXXXXXXXXXX'
access_secret = 'XXXXXXXXXXXXXXXXXXXXX'
```

```
class TweetListener(StreamListener):
```

```
# A listener handles tweets are the received from the stream.
```

```
# This is a basic listener that just prints received tweets
to standard output
```

```
def on_data(self, data):
    print data
    return True
```

```
def on_error(self, status):
```

```
    print status
```

```
#printing all the tweets to the standard output
```

```
auth = OAuthHandler(consumer_key, consumer_secret)
auth.set_access_token(access_token, access_secret)
```

```
stream = Stream(auth, TweetListener())
stream.filter(track=['research'])
```

Using this Python code, the keyword 'research' is extracted from Twitter and the output is sent in JSON format. JSON (JavaScript Object Notation) is a special file format that is used by many NoSQL and unstructured data handling engines. Once the JSON is obtained, the required knowledge can be extracted by using the Refine or any other tool, and further predictions can be made. 

By: Dr Gaurav Kumar

The author is the managing director of Magma Research and Consultancy Pvt Ltd, Ambala. He is associated with a number of academic institutes, where he delivers lectures and conducts technical workshops on the latest technologies and tools. E-mail: kumargaurav.in@gmail.com